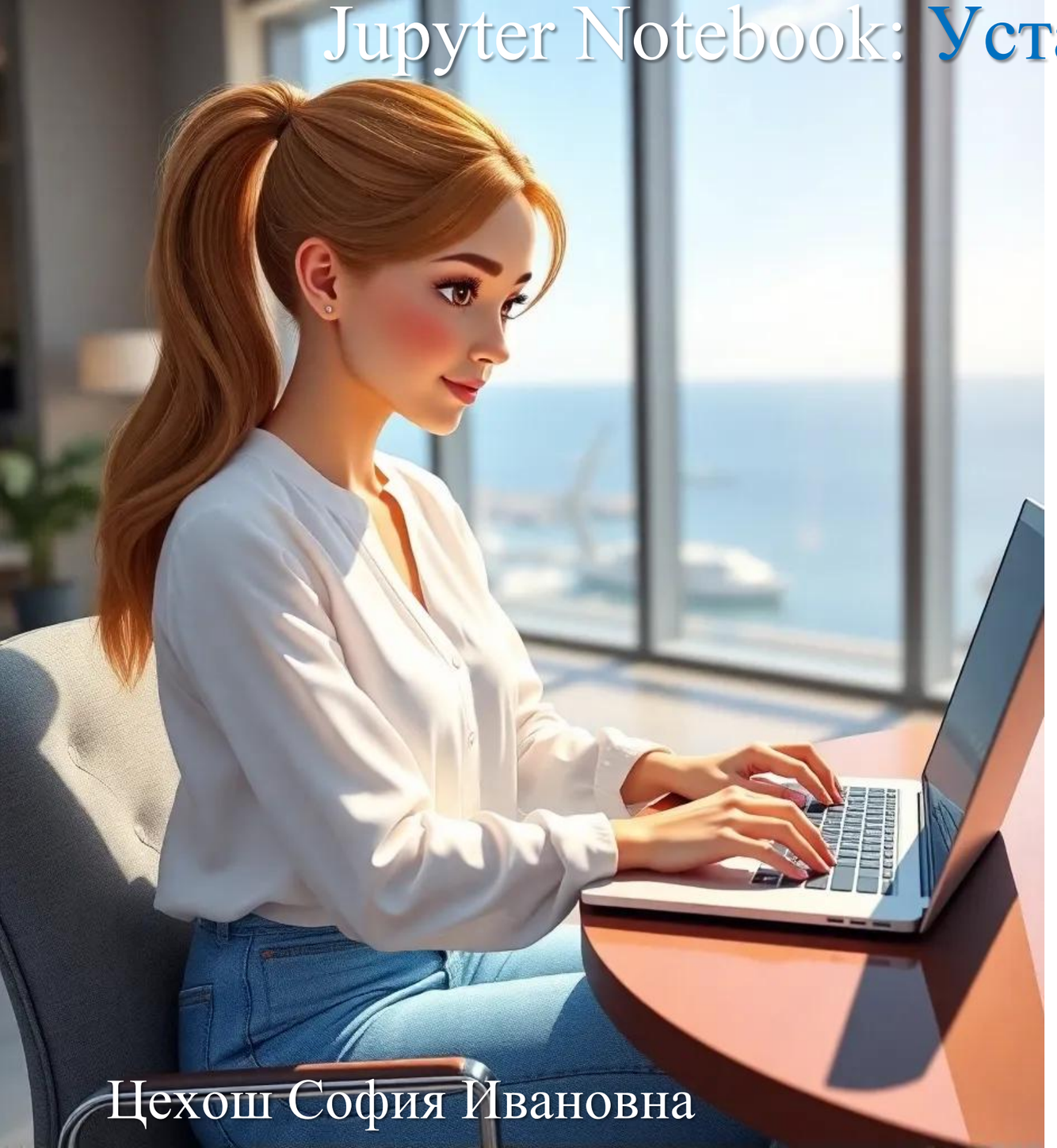
Jupyter Notebook: Установка и Использование

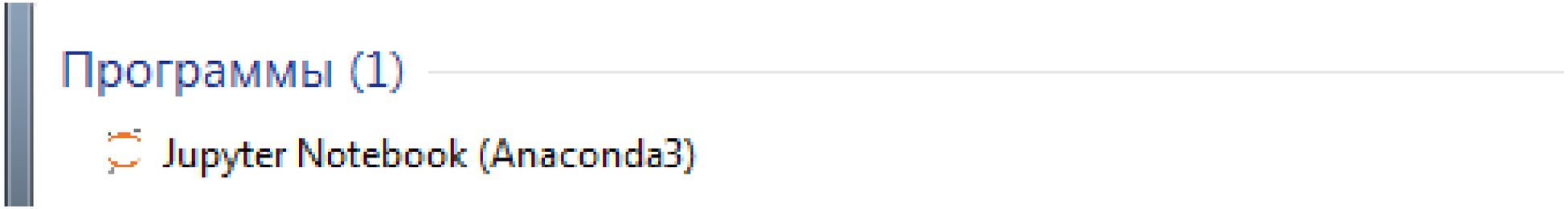


Цехош София Ивановна

Дата обращения 16.11.2025

Проверка установки ПО

1. В меню “Пуск” начните вводить Jupyter



2. Запустите её и проверьте браузер

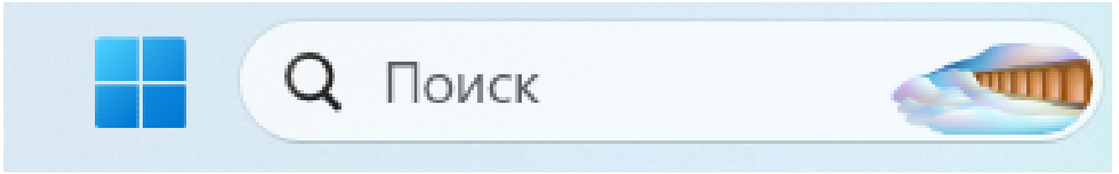
3. Если не открылось окно, то сами перейдите на адрес: http://localhost:8888/tree

4. Если Jupyter’а нет, качаем:

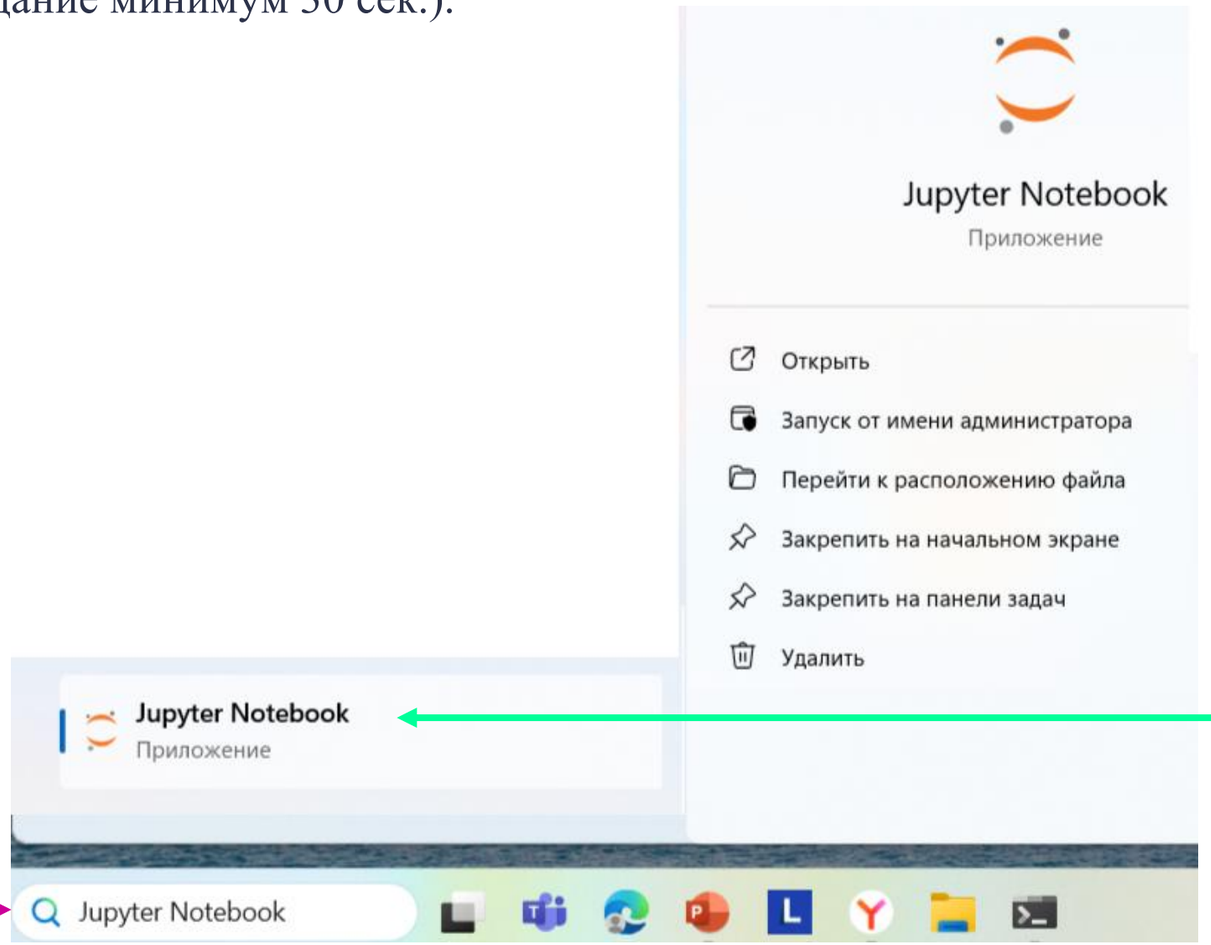
ссылка: https://disk.yandex.ru/d/Ag--V3Y2-X7iBQ

Создание и запуск Jupyter Notebook

1 Откройте поисковую строку или терминал.



2 Введите **jupyter notebook** и нажмите Enter. Если появится Jupyter Notebook кликните на иконку приложения. (ожидание минимум 30 сек.).

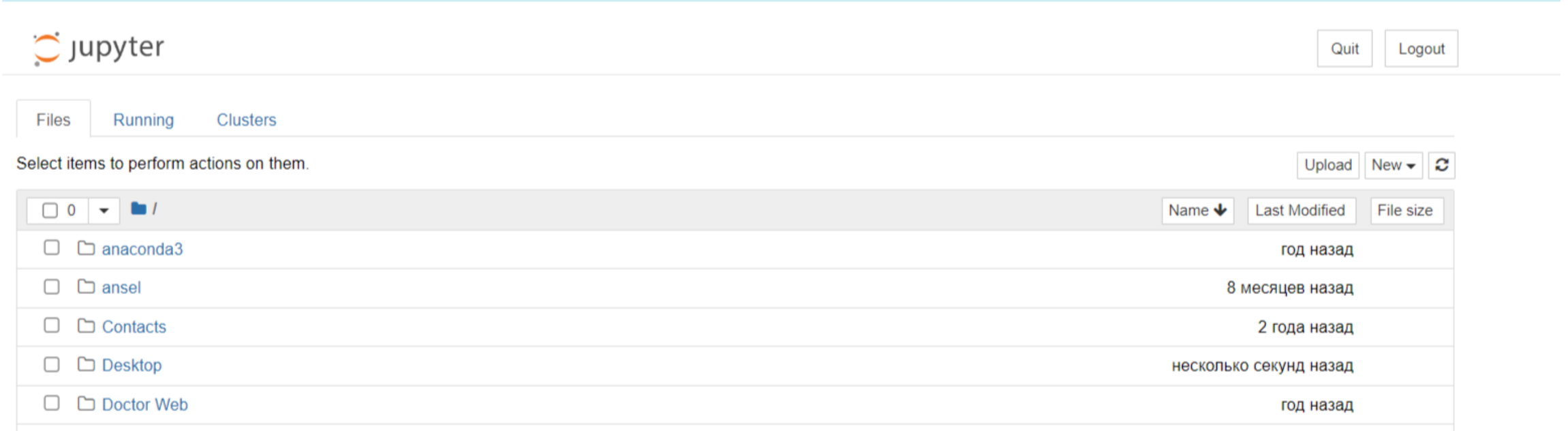


Создание и запуск Jupyter Notebook

3 Если не открылось окно, то сами перейдите на адрес: http://localhost:8888/tree

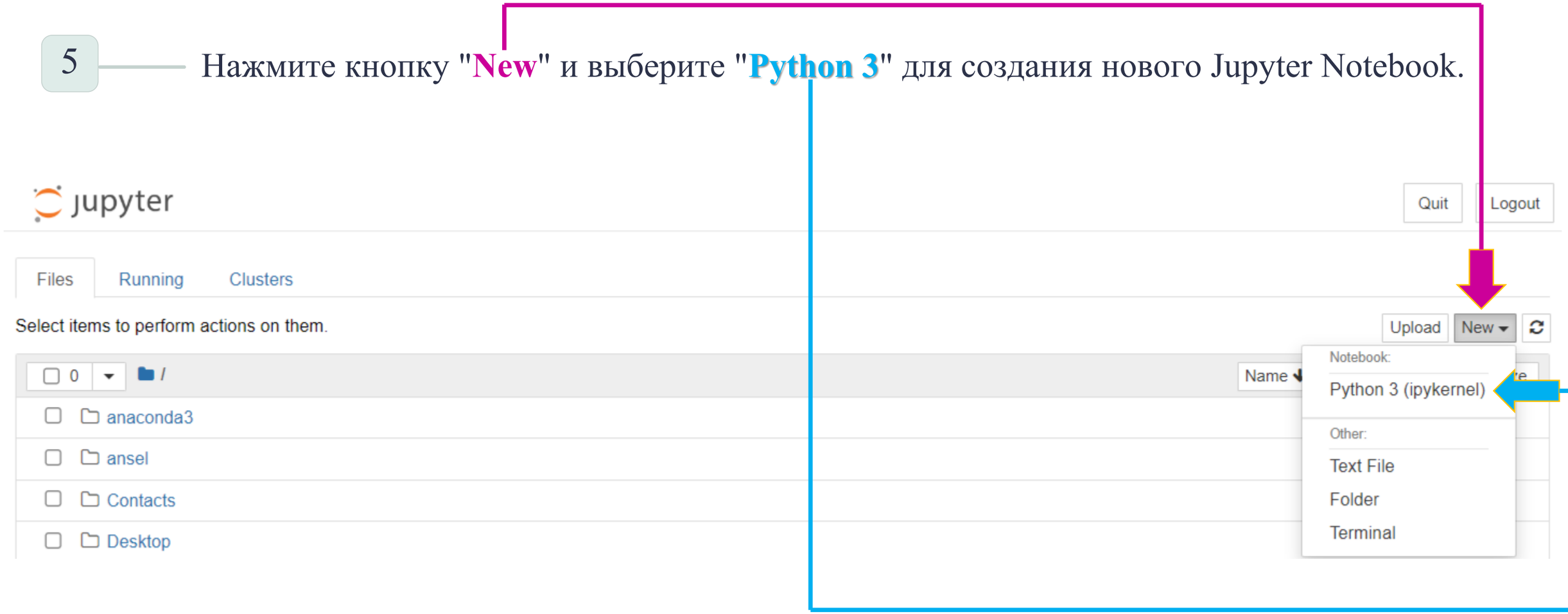
Создание и запуск Jupyter Notebook

4 Стартовая страница программы Jupyter Notebook.



Создание и запуск Jupyter Notebook

5 Нажмите кнопку "**New**" и выберите "**Python 3**" для создания нового Jupyter Notebook.



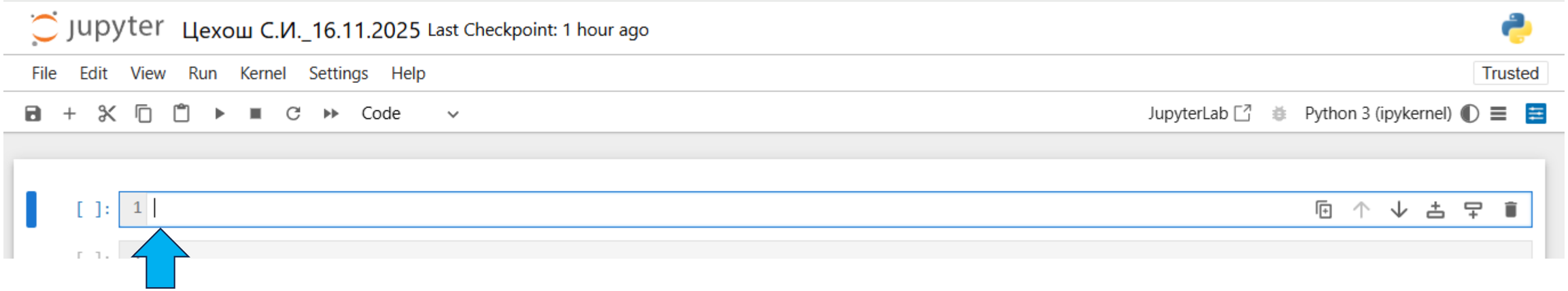
Создание и запуск Jupyter Notebook

5 Создан новый пустой файл Python 3 (ipykernel).

Код следует вводить в ячейки типа **'Code',** обозначенные как **[ ]:**



После [ ]: необходимо поставить курсор для ввода кода.



После [ ]: осуществляем ввод кода.

```
[ ]: print('Введите команду "start"')

[ ]: user = input()
```

В **одну ячейку** можно ввести несколько строк кода

```
[1]: print('Введите команду "start"')

Введите команду "start"

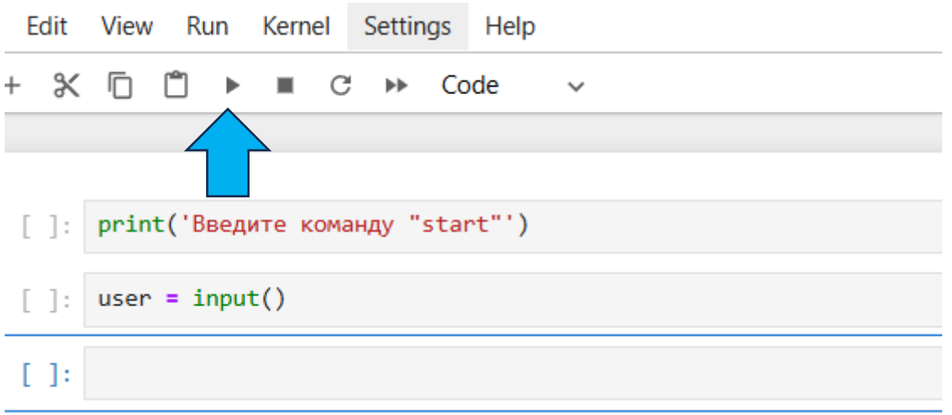
[2]: user = input()

 start

[3]: if user.lower() == 'start':
        print(f'Я Ваш помощник.\n[1] - Добавить товар \n[2] - Посмотреть список товаров \n[stop] - команда "Завершить работу программы"')

Я Ваш помощник.
[1] - Добавить товар
[2] - Посмотреть список товаров
[stop] - команда "Завершить работу программы"
```

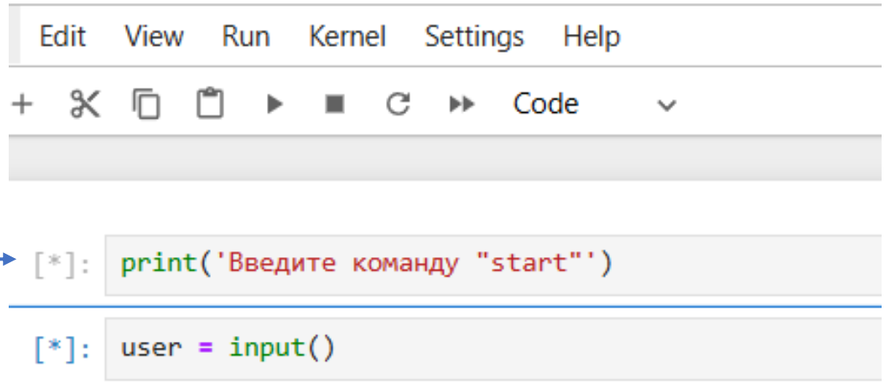
Правила выполнения:



▶️ **Run** - Выполняет код в текущей выбранной ячейке.

Переходит к следующей ячейке после выполнения.

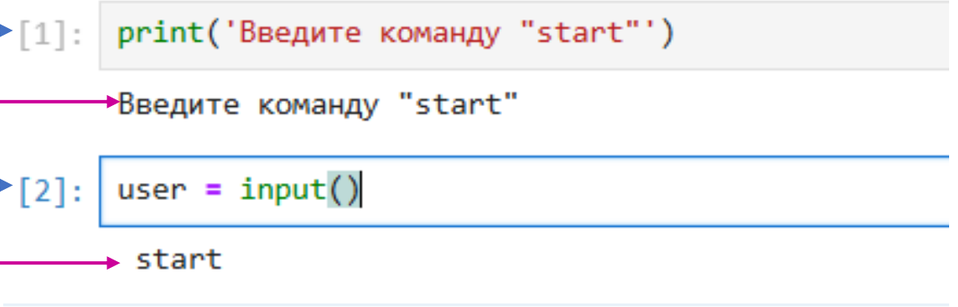
Символ [*] – это может указывать на то, что **ячейка сейчас выполняется**, будет выполнена, когда придёт её очередь, или не выполняется, потому что ядро было перезапущено или прервано



[1] – **числа** в [ ] отображают порядок текущего выполнения кода

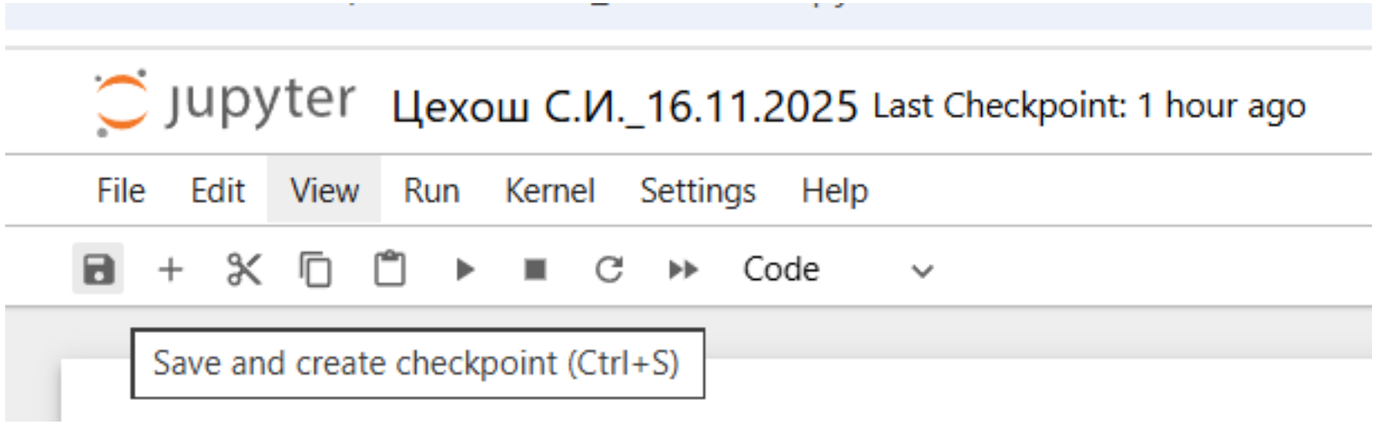
Ждите появления номера в скобках [1] и т.д.

Результат появится под ячейкой

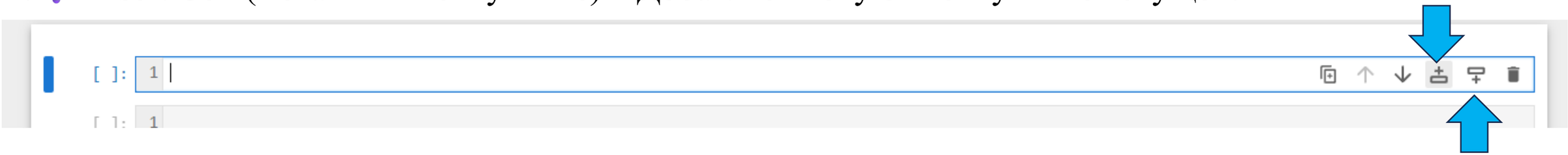


Инструкция по использованию основных кнопок панели Jupyter Notebook:

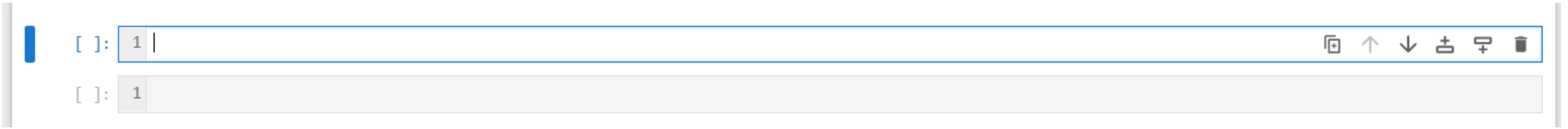
1. 💾 **Save (Сохранить) -** Сохраняет текущий notebook



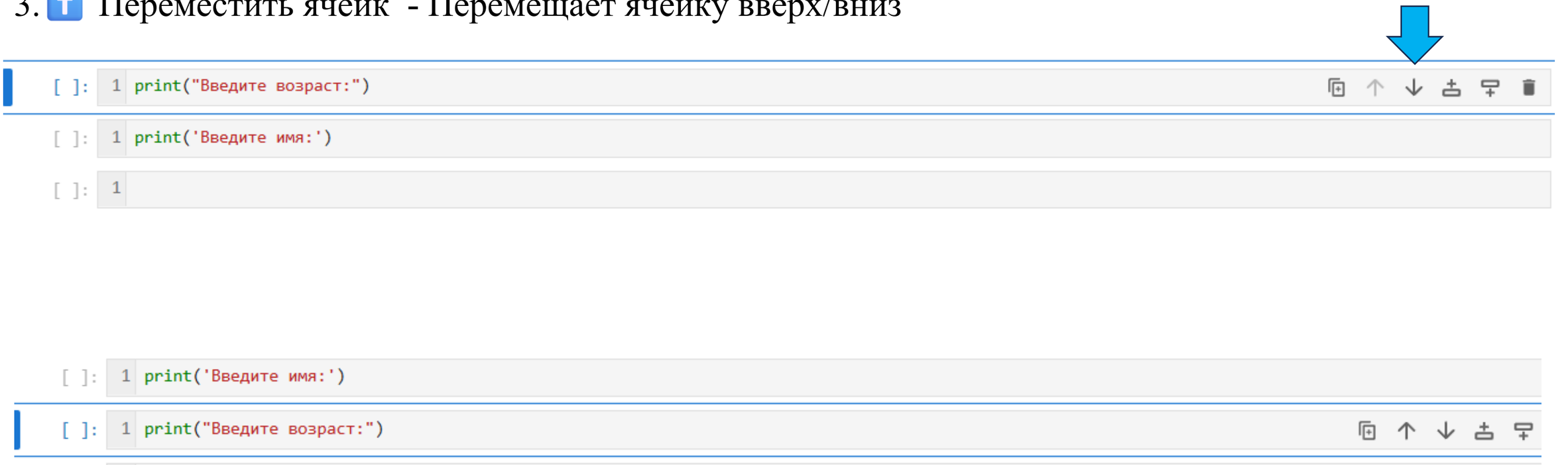
2. ➕ Insert Cell (Вставить ячейку ниже) – Добавляет новую ячейку ниже текущей.



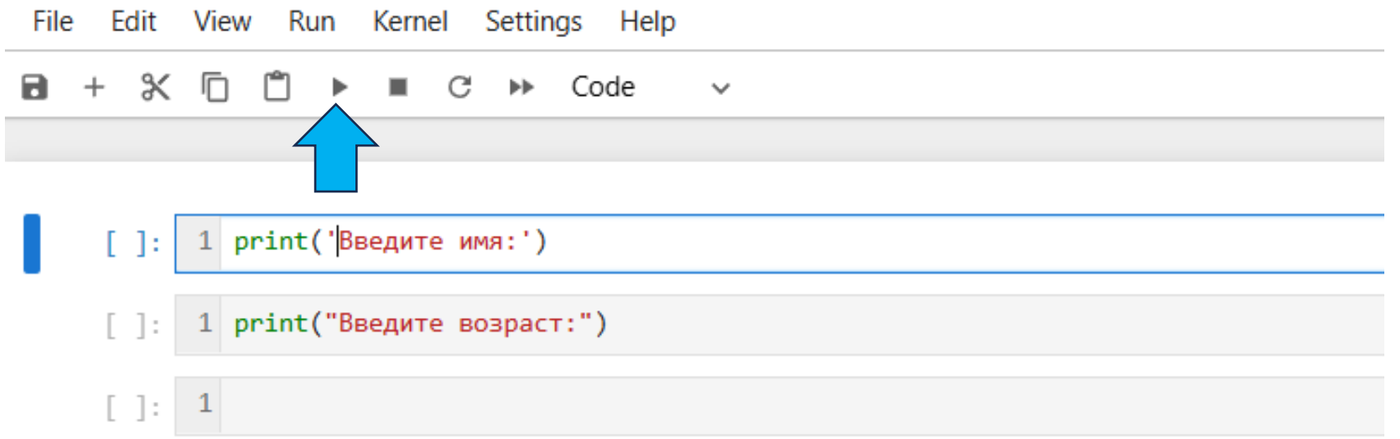
Горячая клавиша: B (ниже), A (выше)

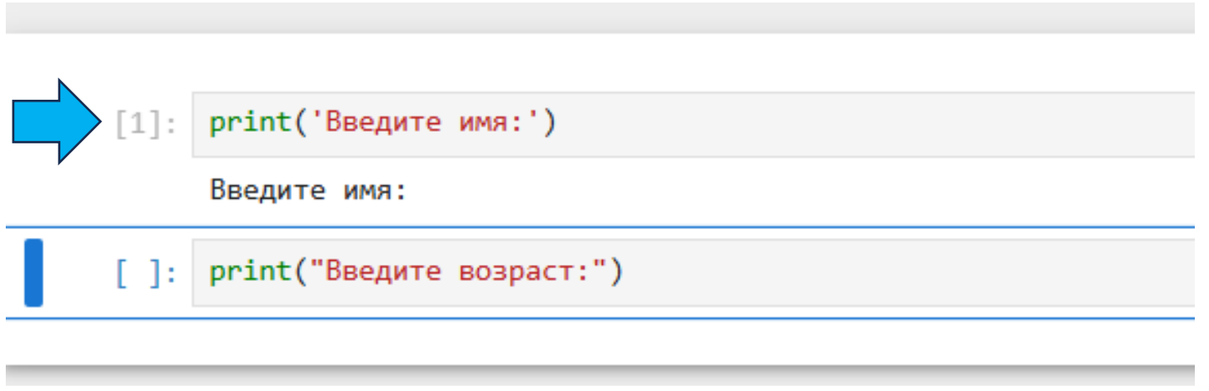


3. ⬆️ Переместить ячейк - Перемещает ячейку вверх/вниз



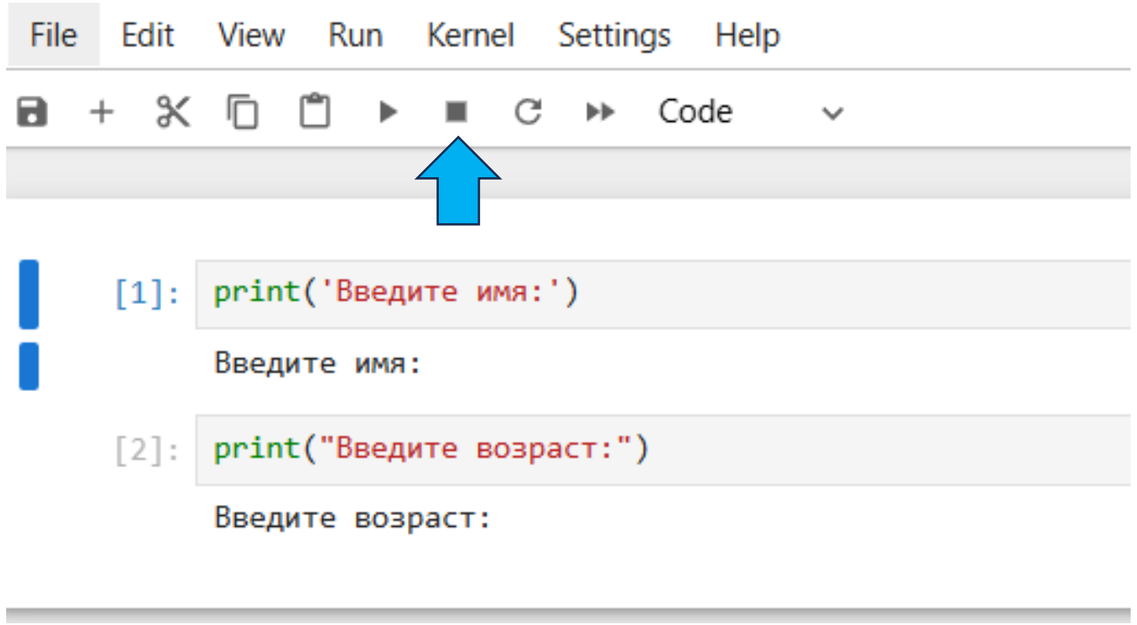
4. ▶️ Run - Выполняет код в текущей выбранной ячейке. Переходит к следующей ячейке после выполнения.





5. ⏹ Stop (Остановить) - Останавливает выполнение кода.

Кнопка полезна при зависании или бесконечных циклах.

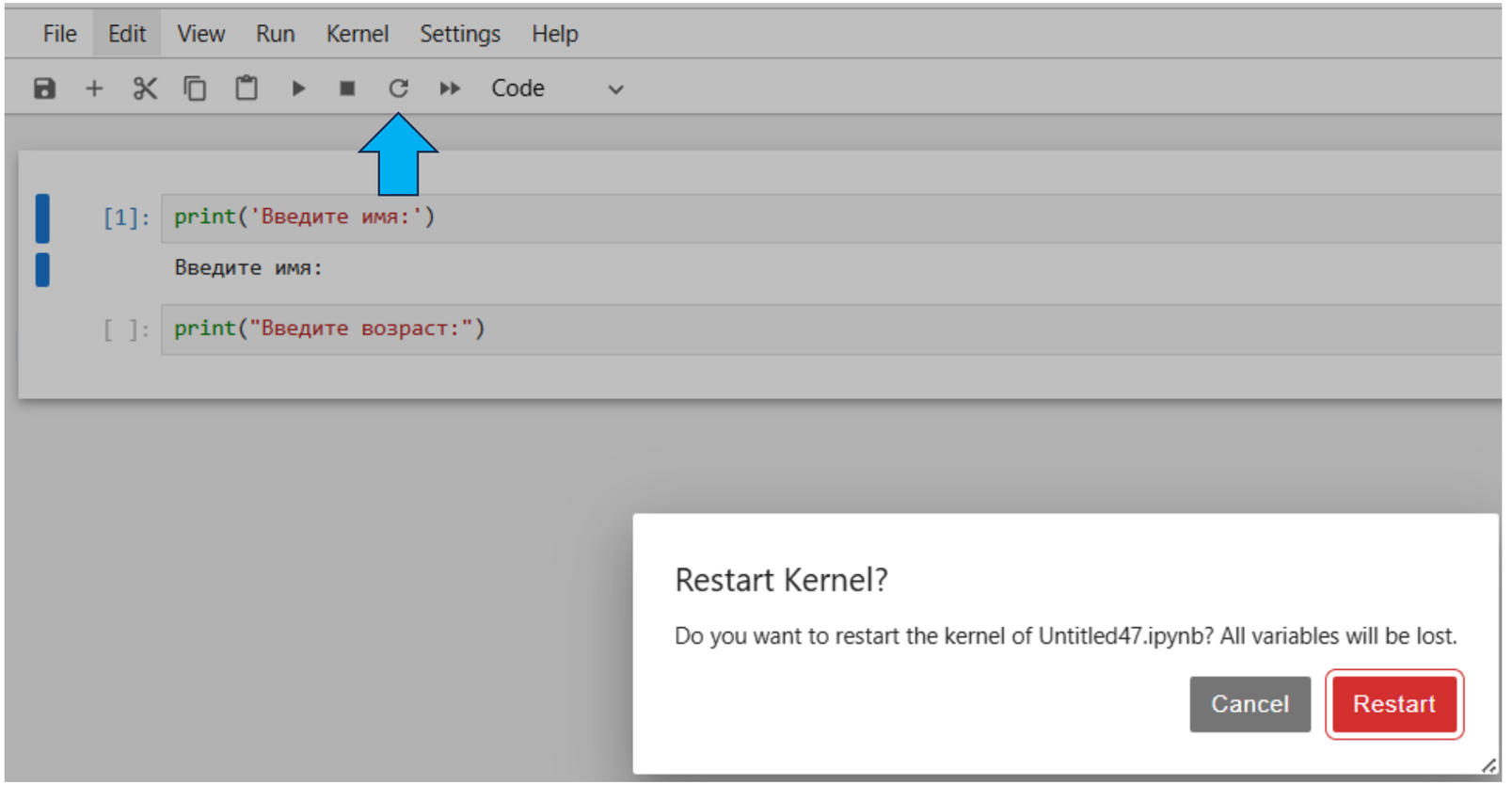


6. 🔄 Перезапустить ядро - Перезапускает Python kernel (ядро Python)

Очищает все переменные из памяти.

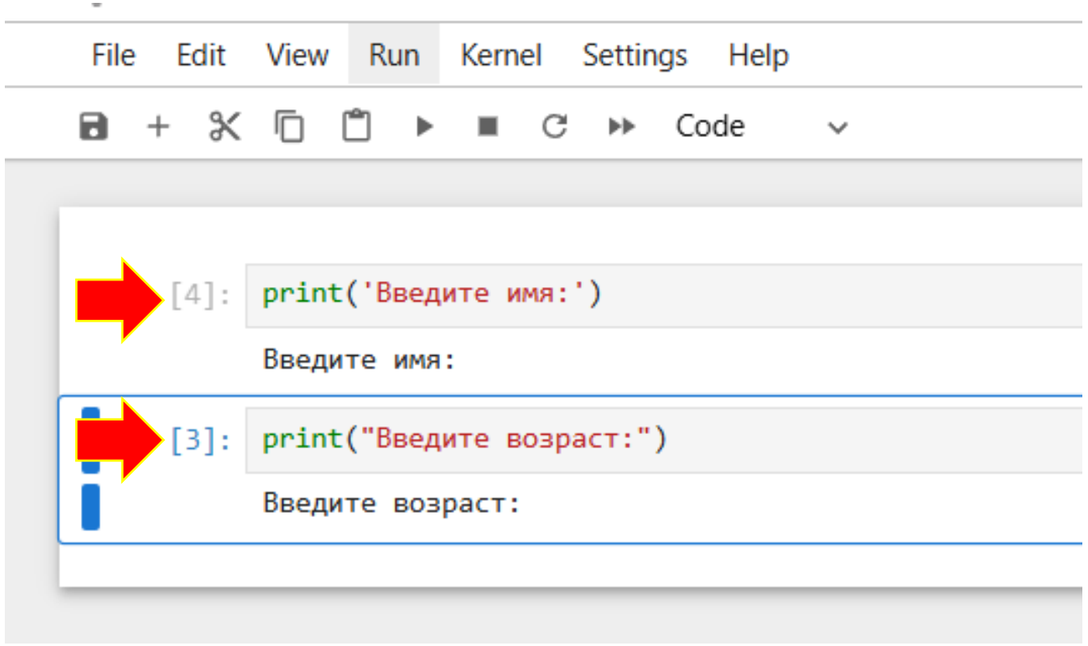
Не удаляет код из ячеек.

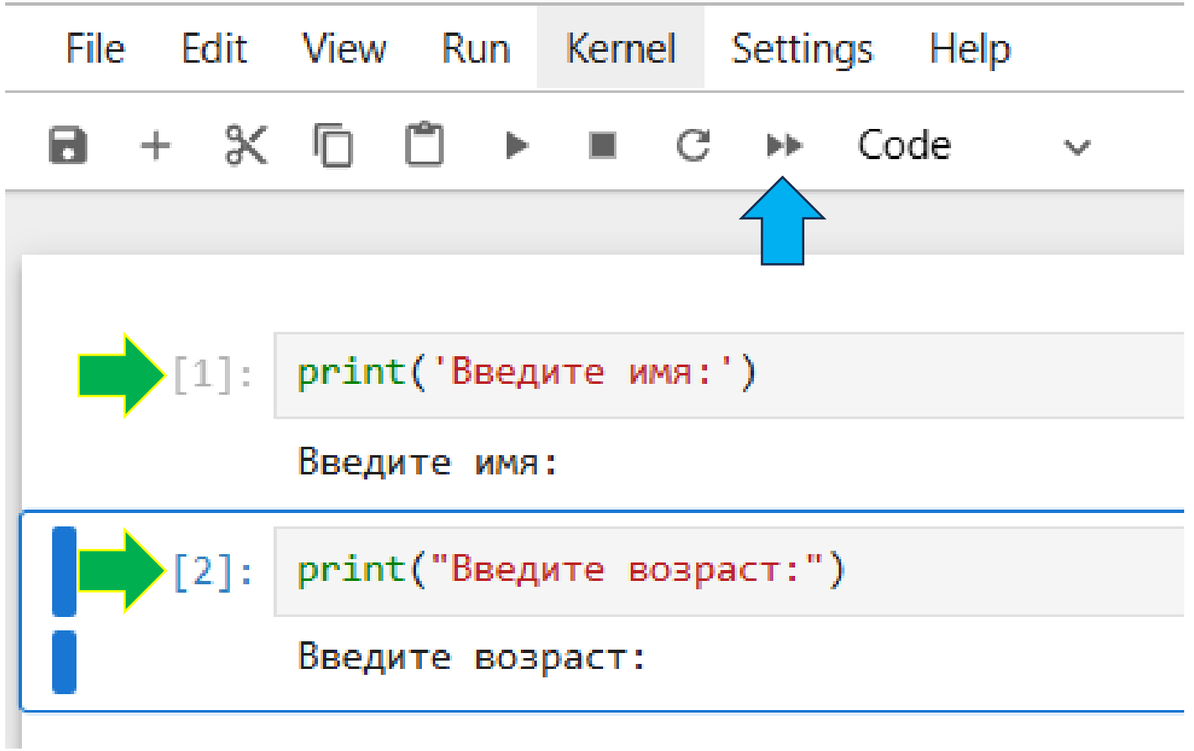
Требует подтверждения действия.



Инструкция по использованию основных кнопок панели Jupyter Notebook:

7. ⏩ подтвердите перезапуск кода и запустите все ячейки

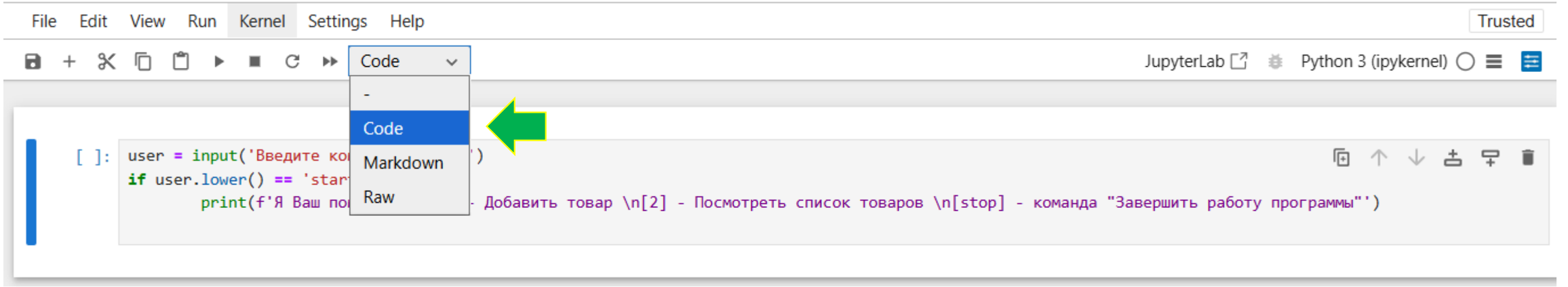




Code (Код)

Используется для написания и выполнения программного кода.

Поддерживает подсветку синтаксиса Python.




```
[1]: user = input('Введите команду "start"')
     if user.lower() == 'start':
             print(f'Я Ваш помощник.\n[1] - Добавить товар \n[2] - Посмотреть список товаров \n[stop] - команда "Завершить работу программы"')

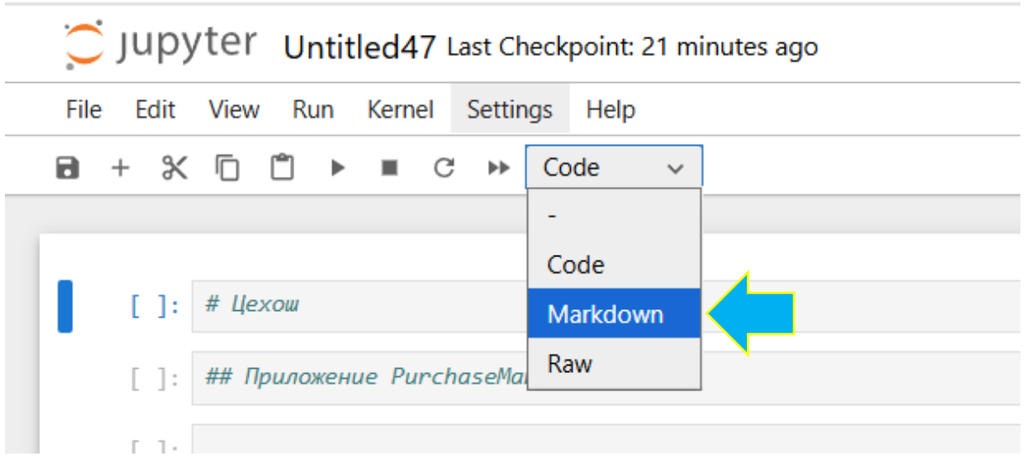
     Введите команду "start" start
     Я Ваш помощник.
     [1] - Добавить товар
     [2] - Посмотреть список товаров
     [stop] - команда "Завершить работу программы"
```

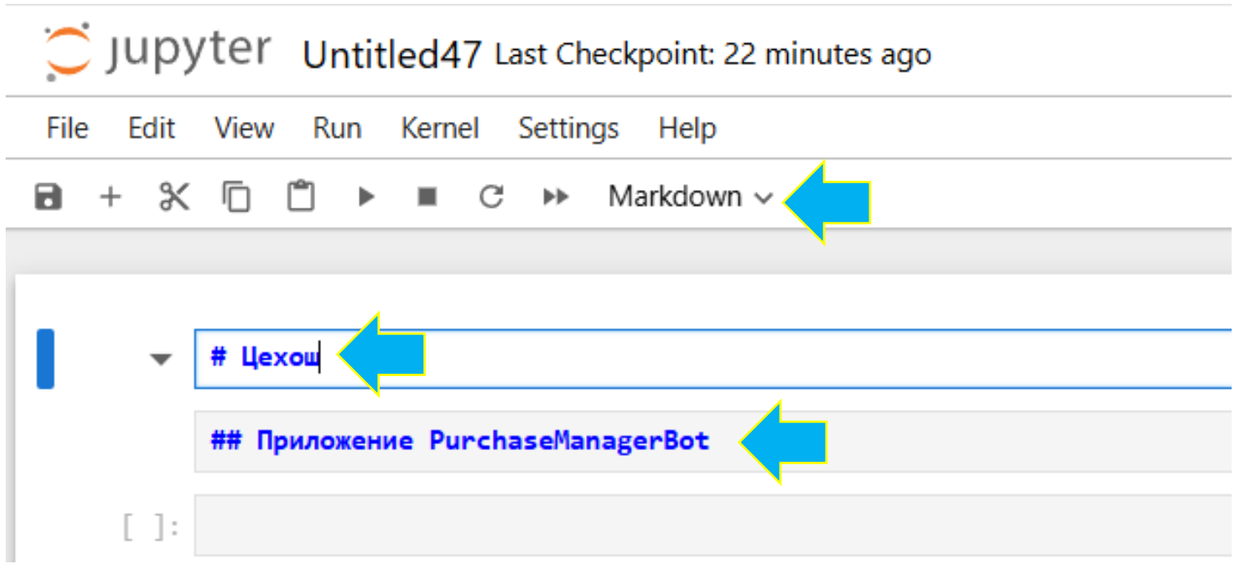
Markdown (Заголовок)заголовки создаются с помощью ячеек Markdown и символа решётки (#):

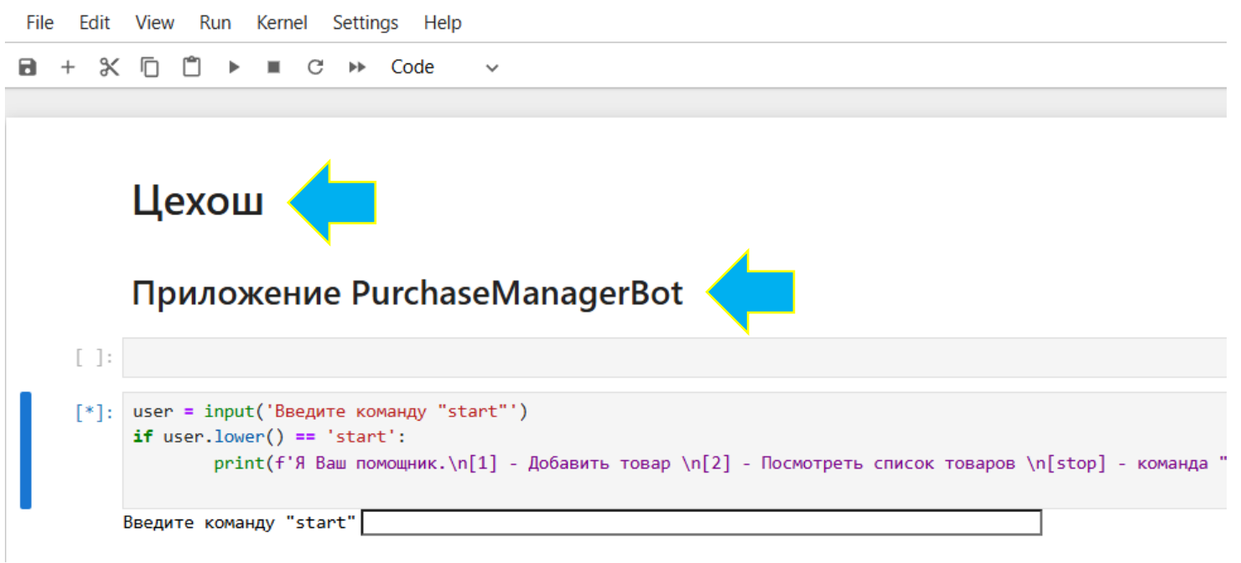
— заголовок первого уровня (самый крупный)

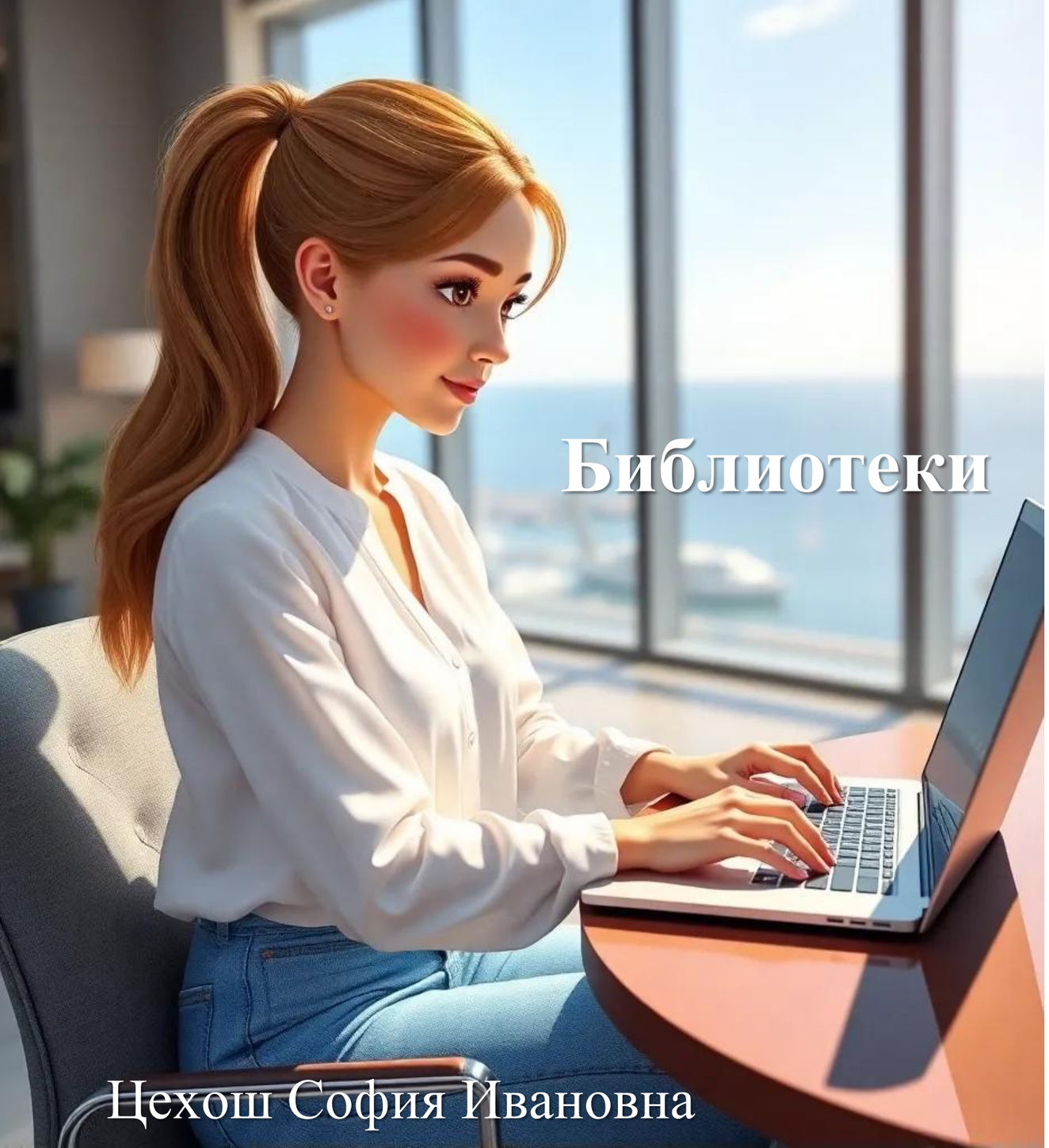
— заголовок второго уровня

— заголовок третьего уровня









Библиотеки

Цехош София Ивановна

Дата обращения 16.11.2025

Библиотека **import pyttsx3** для преобразования **текста в речь**

```
pip install pyttsx3==2.98  # Установка конкретной версии библиотеки для синтеза речи
import pyttsx3
engine = pyttsx3.init() # Инициализация движка синтеза речи
engine.setProperty("rate", 180) # Настройка скорости речи (слов в минуту)
engine.setProperty("volume", 0.5) # Настройка громкости (0.0 - тишина, 1.0 - максимум)
engine.say("Запиши фразу") # Добавление фразы в очередь на произношение
engine.runAndWait() # Воспроизведение речи и ожидание завершения
```

Преимущества pyttsx3:

✅ Простая установка (pip install pyttsx3)

✅ Не требует интернета

✅ Бесплатная

✅ Поддержка русского языка (**Использует системные голосовые движки**)

✅ Настройка голоса и скорости

pip - менеджер пакетов Python

install - используется для установки библиотек и зависимостей

pyttsx3 -команда для установки пакета

==2.98 - название библиотеки для преобразования текста в речь

Указание **конкретной версии** для установки

Без == установилась бы последняя версия

Библиотека **import pyttsx3** . Пример

Код:

```
[ ]: pip install pyttsx3==2.98
```

```
[ ]: import pyttsx3
```

```
[ ]: engine = pyttsx3.init()# Инициализация модуля синтеза речи
     engine.setProperty("rate", 180)# Установка скорости воспроизведения (слов в минуту)
     engine.setProperty("volume", 0.5)# Настройка уровня громкости (диапазон: 0.0-1.0)
     engine.say("Введите Ваше имя")# преобразование текста в речь
     engine.runAndWait()
     user_name = input('Введите Ваше имя:').title()
     print(user_name)
```

Исполнение кода:

```
[1]: pip install pyttsx3==2.98

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pyttsx3==2.98 in c:\users\paw23\appdata\roaming\python\python312\site-packages (2.98)
Requirement already satisfied: comtypes in c:\users\paw23\appdata\roaming\python\python312\site-packages (from pyttsx3==2.98) (1.4.13)
Requirement already satisfied: pypiwin32 in c:\users\paw23\appdata\roaming\python\python312\site-packages (from pyttsx3==2.98) (223)
Requirement already satisfied: pywin32 in c:\programdata\anaconda3\anaconda3_77\lib\site-packages (from pyttsx3==2.98) (305.1)
Note: you may need to restart the kernel to use updated packages.
```

```
[2]: import pyttsx3
```

```
[3]: engine = pyttsx3.init()# Инициализация модуля синтеза речи
     engine.setProperty("rate", 180)# Установка скорости воспроизведения (слов в минуту)
     engine.setProperty("volume", 0.5)# Настройка уровня громкости (диапазон: 0.0-1.0)
     engine.say("Введите Ваше имя")# преобразование текста в речь
     engine.runAndWait()
     user_name = input('Введите Ваше имя:').title()
     print(user_name)

Введите Ваше имя: sofia
Sofia
```

Библиотека **import pyttsx3** . Пример

[1]:
```
pip install pyttsx3==2.98
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pyttsx3==2.98 in c:\users\paw23\appdata\roaming\python\python312\site-packages (2.98)
Requirement already satisfied: comtypes in c:\users\paw23\appdata\roaming\python\python312\site-packages (from pyttsx3==2.98) (1.4.13)
Requirement already satisfied: pypiwin32 in c:\users\paw23\appdata\roaming\python\python312\site-packages (from pyttsx3==2.98) (223)
Requirement already satisfied: pywin32 in c:\programdata\anaconda3\anaconda3_77\lib\site-packages (from pyttsx3==2.98) (305.1)
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import pyttsx3
engine = pyttsx3.init()
engine.setProperty("rate", 180)
engine.setProperty("volume", 0.5)
```

[*]:
```
shopping_list = []
price_list = []

engine.say("Для начала работы введите команду start:")
engine.runAndWait()
start = input('Для начала работы введите команду start:\n')

if start.lower() == 'start':
    engine.say("Введите Ваше имя:")
    engine.runAndWait()
    user_input_name = input('Введите Ваше имя:').title()
    while True:
        print('Я Ваш помощник \n[1] - Добавить товар \n[2] - Посмотреть список товаров \n[stop] - команда "Завершить работу программы"' )
        user_input = input('Введите команду: ')
        if user_input == '1':
            flag = True
            while flag:
                engine.say("Введите название товара:")
                engine.runAndWait()
                shopping = input('Введите название товара: ').lower()
                print(shopping_list)

```

```
Для начала работы введите команду start:
 start
Введите Ваше имя: sofia
Я Ваш помощник
[1] - Добавить товар
[2] - Посмотреть список товаров
[stop] - команда "Завершить работу программы"
Введите команду:  1
Введите название товара:  молоко
```

Библиотека для построения графиков import **matplotlib.pyplot as plt**

Типы графиков:

plt.**plot()** - линейный график (изменение во времени/порядке)

plt.**bar()** - гистограмма - столбчатая диаграмма (сравнение величин)

plt.**pie()** - круговая диаграмма (доли от целого)

plt.**scatter() -** точечная диаграмма (корреляция данных)

Библиотека для построения графиков import **matplotlib.pyplot as plt**

import **matplotlib.pyplot as plt**

plt.figure(figsize=(10, 5)) # Создаем область для графика размером 10x5 дюймов

plt.**plot**(название списка, название списка, marker='o', linewidth=2) # Строим график: X-товары, Y-цены, точки

plt.title(" Укажите заголовок") # Заголовок графика

plt.xlabel("Название, ед. измерения") # Подпись оси X

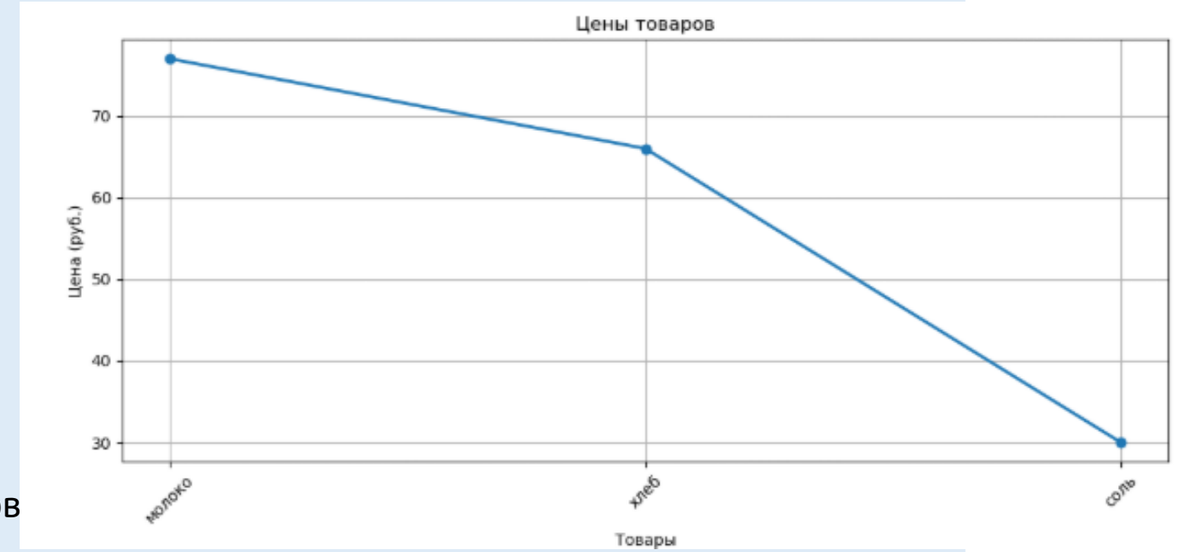
plt.ylabel("Название, ед. измерения") # Подпись оси Y

plt.grid(True) # Включаем сетку

plt.xticks(rotation=45) # Поворачиваем подписи на оси X на 45 градусов

plt.tight_layout() # Автоматически настраиваем расположение элементов

plt.show() # Показываем график



ПРИМЕР (линейный график - plt.**plot**)

Код

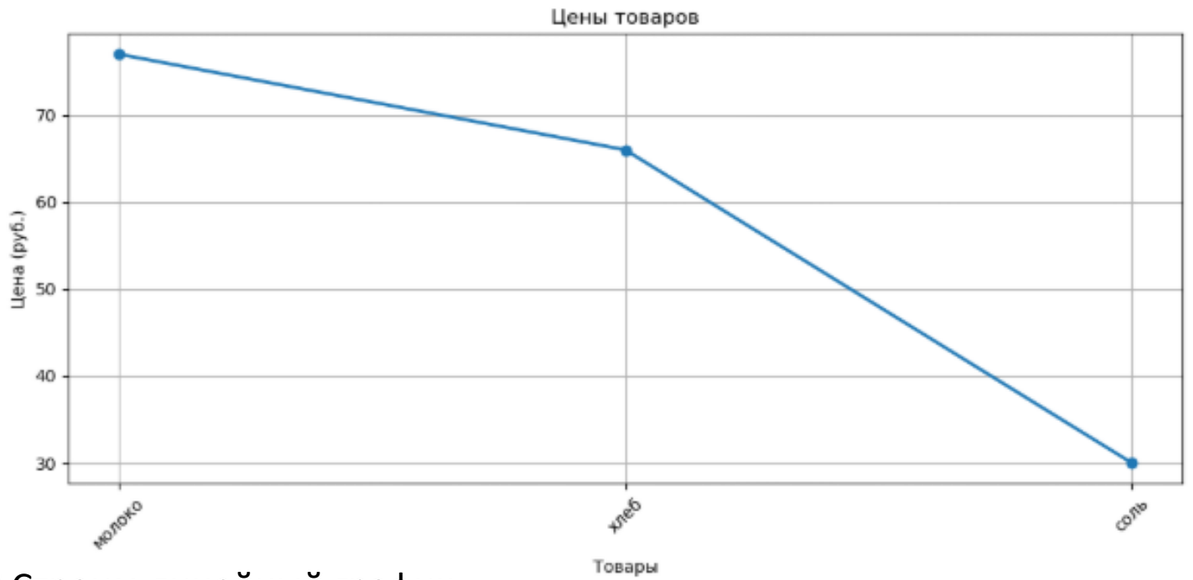
```
[1]: 1 import matplotlib.pyplot as plt
```

```
[*]:
shopping_list = []
price_list = []
start = input('Для начала работы введите команду start:\n')
if start.lower() == 'start':
    user_input_name = input('Введите Ваше имя:').title()
    while True:
        print('Я Ваш помощник \n[1] - Добавить товар \n[2] - Аналитика цен ')
        user_input = input('Введите команду: ')
        if user_input == '1':
            flag = True
            while flag:
                shopping = input('Введите название товара: ').lower()
                shopping_list.append(shopping)
                price = float(input('Введите цену товара: '))
                price_list.append(price)
                print(f'Добавлен товар: {shopping} - {price} руб.')
                user_2 = input('Хотите добавить ещё товар? [1] - Да [2] - Нет\n')
                if user_2 == '1':
                    flag = True
                else:
                    flag = False
        elif user_input == '2':
                    print('\nВаш список покупок:')
                    for i in range(len(shopping_list)):
                        print(f'{i+1}. {shopping_list[i]} - {price_list[i]} руб.')
                    # Строим график
                    plt.figure(figsize=(10, 5))
                    plt.plot(shopping_list, price_list, marker='o', linewidth=2)
                    plt.title("Цены товаров")
                    plt.xlabel("Товары")
                    plt.ylabel("Цена (руб.)")
                    plt.grid(True)
                    plt.xticks(rotation=45)
                    plt.tight_layout()
                    plt.show()
```

Вывод

```
                    plt.tight_layout()
                    plt.show()
Для начала работы введите команду start:
 start
Введите Ваше имя: sofia
Я Ваш помощник
[1] - Добавить товар
[2] - Аналитика цен
Введите команду:  1
Введите название товара:  молоко
Введите цену товара:  77
Добавлен товар: молоко - 77.0 руб.
Хотите добавить ещё товар? [1] - Да [2] - Нет
 1
Введите название товара:  хлеб
Введите цену товара:  66
Добавлен товар: хлеб - 66.0 руб.
Хотите добавить ещё товар? [1] - Да [2] - Нет
 1
Введите название товара:  соль
Введите цену товара:  30
Добавлен товар: соль - 30.0 руб.
Хотите добавить ещё товар? [1] - Да [2] - Нет
 2
Я Ваш помощник
[1] - Добавить товар
[2] - Аналитика цен
Введите команду:  2

Ваш список покупок:
1. молоко - 77.0 руб.
2. хлеб - 66.0 руб.
3. соль - 30.0 руб.
```



plt.**plot**(shopping_list, price_list**, marker='o', linewidth=2**) # Строим линейный график

ПРИМЕР (гистограмма plt.bar)

Код

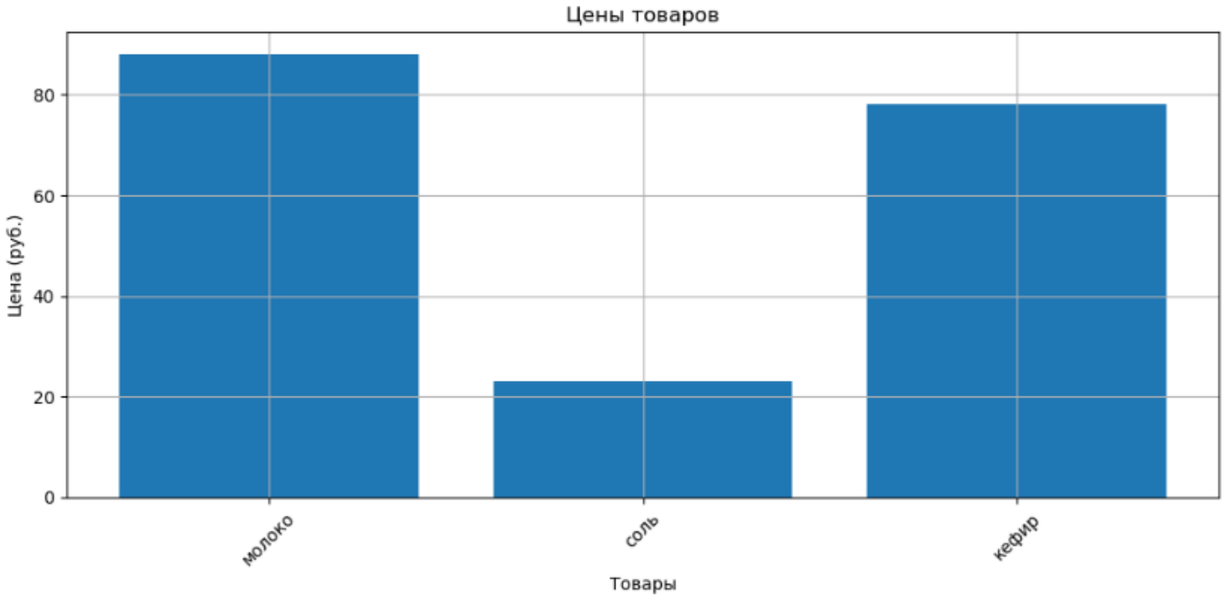
```
[1]: 1 import matplotlib.pyplot as plt
```

```
[*]:
shopping_list = []
price_list = []
while True:
    print('Я Ваш помощник \n[1] - Добавить товар \n[2] - Аналитика цен ')
    user_input = input('Введите команду: ')
    if user_input == '1':
        flag = True
        while flag:
            shopping = input('Введите название товара: ').lower()
            shopping_list.append(shopping)
            price = float(input('Введите цену товара: '))
            price_list.append(price)
            print(f'Добавлен товар: {shopping} - {price} руб.')
            user_2 = input('Хотите добавить ещё товар? [1] - Да [2] - Нет\n')
            if user_2 == '1':
                flag = True
            else:
                flag = False
    elif user_input == '2':
                print('\nВаш список покупок:')
                plt.figure(figsize=(10, 5))
                plt.bar(shopping_list, price_list)# Строим гистограмму
                plt.title("Цены товаров")
                plt.xlabel("Товары")
                plt.ylabel("Цена (руб.)")
                plt.grid(True)
                plt.xticks(rotation=45)
                plt.tight_layout()
                plt.show()
```

Вывод

```
Я Ваш помощник
[1] - Добавить товар
[2] - Аналитика цен
Введите команду:  1
Введите название товара:  молоко
Введите цену товара:  88
Добавлен товар: молоко - 88.0 руб.
Хотите добавить ещё товар? [1] - Да [2] - Нет
 1
Введите название товара:  соль
Введите цену товара:  23
Добавлен товар: соль - 23.0 руб.
Хотите добавить ещё товар? [1] - Да [2] - Нет
 1
Введите название товара:  кефир
Введите цену товара:  78
Добавлен товар: кефир - 78.0 руб.
Хотите добавить ещё товар? [1] - Да [2] - Нет
 2
Я Ваш помощник
[1] - Добавить товар
[2] - Аналитика цен
Введите команду:  2

Ваш список покупок:
```



plt.**bar(**shopping_list, price_list**) #** Строим гистограмму

ПРИМЕР (гистограмма plt.**bar**)

Код

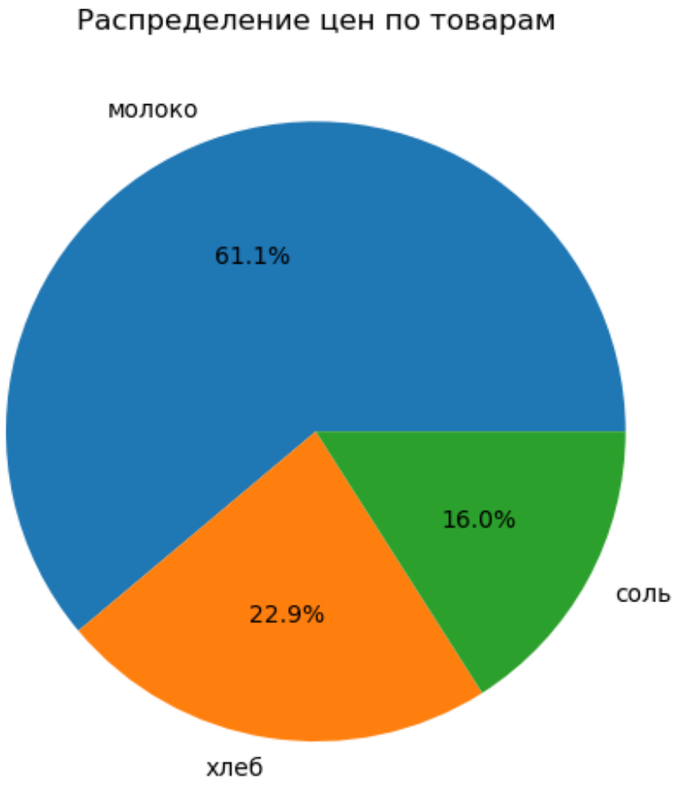
```
[1]: 1 import matplotlib.pyplot as plt
```

```
[*]:
1  shopping_list = []
2  price_list = []
3  while True:
4      print('Я Ваш помощник \n[1] - Добавить товар \n[2] - Аналитика цен ')
5      user_input = input('Введите команду: ')
6      if user_input == '1':
7          flag = True
8          while flag:
9              shopping = input('Введите название товара: ').lower()
10             shopping_list.append(shopping)
11             price = float(input('Введите цену товара: '))
12             price_list.append(price)
13             print(f'Добавлен товар: {shopping} - {price} руб.')
14             user_2 = input('Хотите добавить ещё товар? [1] - Да [2] - Нет\n')
15             if user_2 == '1':
16                 flag = True
17             else:
18                 flag = False
19     elif user_input == '2':
20                 print('\nВаш список покупок:')
21                 for i in range(len(shopping_list)):
22                     print(f'{i+1}. {shopping_list[i]} - {price_list[i]} руб.')
23                     # Строим круговую диаграмму
24                     plt.figure(figsize=(10, 5))
25                     plt.pie(price_list, labels=shopping_list, autopct='%1.1f%%')
26                     plt.title("Распределение цен по товарам")
27                     plt.tight_layout()
28                     plt.show()
```

Вывод

```
 1
Введите название товара:  соль
Введите цену товара:  23
Добавлен товар: соль - 23.0 руб.
Хотите добавить ещё товар? [1] - Да [2] - Нет
 2
Я Ваш помощник
[1] - Добавить товар
[2] - Аналитика цен
Введите команду:  2

Ваш список покупок:
```



plt.**pie(**price_list, labels=shopping_list, **autopct='%1.1f%%')**

from IPython.display import display

```
from IPython.display import display, HTML
import random
def display_image_auto_hide(path, description, delay_ms=10000):
    display(HTML(f'''<div style="display:flex;align-items:center"> <img src="{path}" style="max-width:30%;margin-right:20px">
    <div style="border-left:2px solid #ccc;padding-left:20px">{description}</div></div>
    <script> setTimeout(()=>document.currentScript.parentElement.style.display='none',{delay_ms}) </script>'''))
```

```
display_image_auto_hide("аналитик2.png", "Квалифицированный офтальмолог, специалист по детскому зрению")
```

ОСНОВНЫЕ КОМПОНЕНТЫ:

HTML Структура:

display: flex - современный способ верстки в строку

align-items: center - выравнивание по центру по вертикали

max-width: 30% - изображение занимает не более 30% ширины

border-left - серая вертикальная линия-разделитель

ОСНОВНЫЕ КОМПОНЕНТЫ:

JavaScript Логика:

setTimeout() - таймер задержки

document.currentScript - доступ к текущему скрипту

parentElement - переход к родительскому элементу

Параметры Функции:

path - относительный или абсолютный путь к изображению

description - любой HTML-совместимый текст

delay_ms - время показа (1000 мс = 1 секунда)

ПРИМЕР

Код

```python
from IPython.display import display, HTML
import random

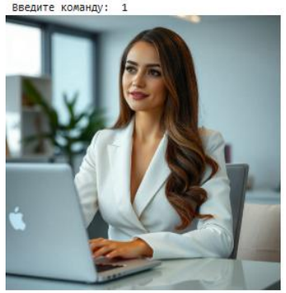
def display_image_auto_hide(path, description, delay_ms=10000):
    display(HTML(f'''<div style="display:flex;align-items:center"> <img src="{path}" style="max-width:30%;margin-right:20px">
    <div style="border-left:2px solid #ccc;padding-left:20px">{description}</div></div>
    <script> setTimeout(()=>document.currentScript.parentElement.style.display='none',{delay_ms}) </script>'''))

while True:
    user_input = input('\n[1] - Наши специалисты\n[2] - Выход\nВведите команду: ')
    if user_input == '1':
        while True:
            user1 = input('\n[1] - Педиатр\n[2] - Офтальмолог\n[3] - Назад\nВведите команду: ')
            if user1 == "1":
                display_image_auto_hide("аналитик1.png", "Опытный педиатр с 10-летним стажем работы")
            elif user1 == "2":
                display_image_auto_hide("аналитик2.png", "Квалифицированный офтальмолог, специалист по детскому зрению")
            elif user1 == "3":
                break
            else:
                print("Неверная команда")
    elif user_input == '2':
            print("До свидания!")
            break  # Этот break выходит из основного цикла while True
```

Вывод

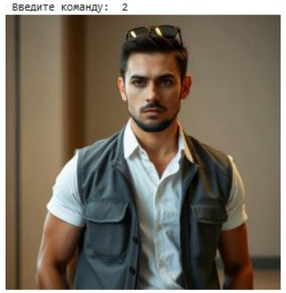
```
[1] - Наши специалисты
[2] - Выход
Введите команду:  1

[1] - Педиатр
[2] - Офтальмолог
[3] - Назад
Введите команду:  1
```



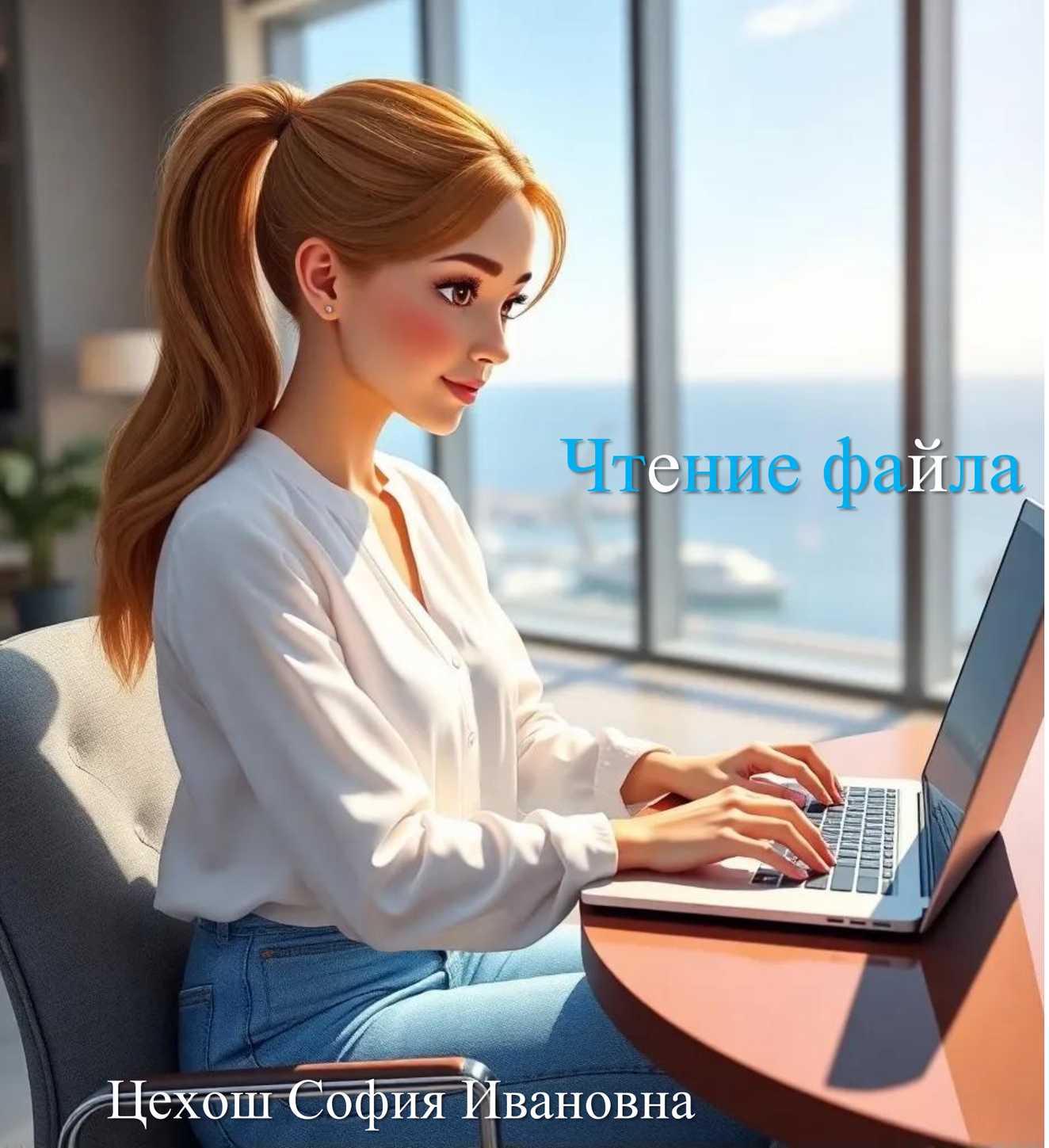
Опытный педиатр с 10-летним стажем работы

```
[1] - Педиатр
[2] - Офтальмолог
[3] - Назад
Введите команду:  2
```



Квалифицированный офтальмолог, специалист по детскому зрению

```
[1] - Педиатр
[2] - Офтальмолог
[3] - Назад
```



Чтение файла

Цехош София Ивановна

Дата обращения 16.11.2025

Чтение содержимого файла

```
file = open('путь к файлу', "r", encoding="utf-8")
text = file.read()
print(text)
file.close()
```

Открытие файла

Для открытия файлов используется функция **open()**.

Функция принимает два основных аргумента:

1. Путь к файлу (относительный или абсолютный).
2. Режим открытия:

'**r**' — читать (read),

'**w**' — записывать,

'**a**' — добавлять.

3. encoding="utf-8" – кодировка определяет, как символы хранятся в файле.

Закрытие файла

После завершения работы с файлом, его необходимо закрыть, используя метод **close()** объекта файла.

Чтение содержимого файла

Метод **readline()**

Читает одну строку из файла.

```
file=open('путь к файлу','r',encoding='utf-8')
text = file.readline()
print(text)
file.close()
```

Метод **readlines()**

Читает все строки из файла и возвращает список строк.

```
file=open ('путь к файлу', 'r',encoding='utf-8')
text = file.readlines()
print(text)
file.close()
```

read() - читает весь файл как одну строку.

readlines() - читает файл и возвращает список строк.

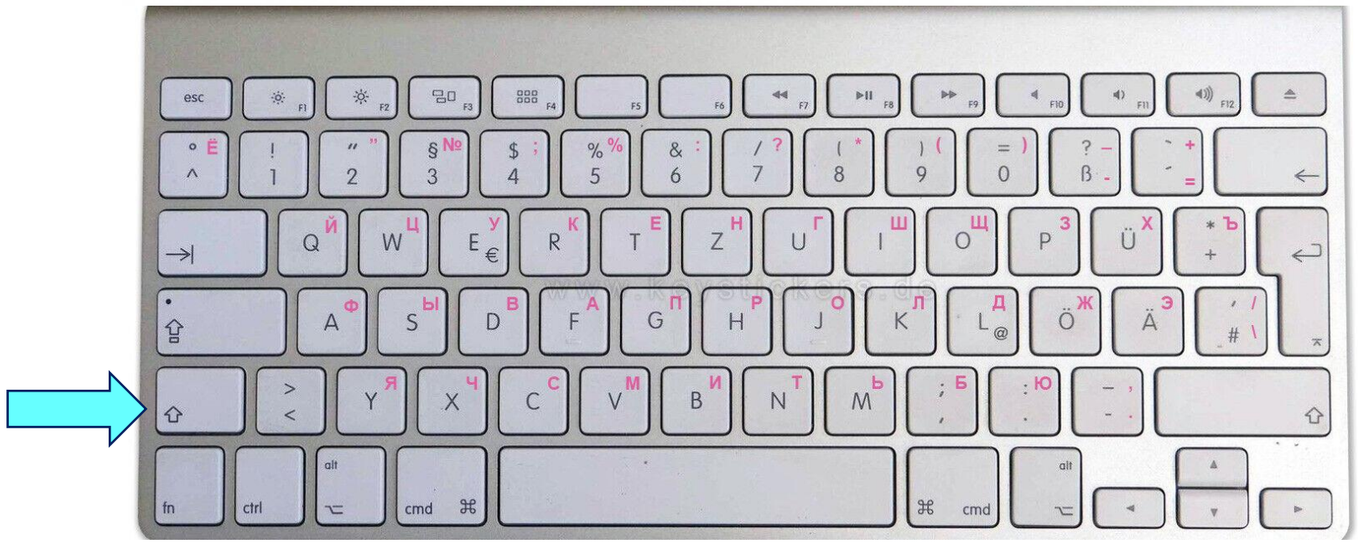
Абсолютный путь

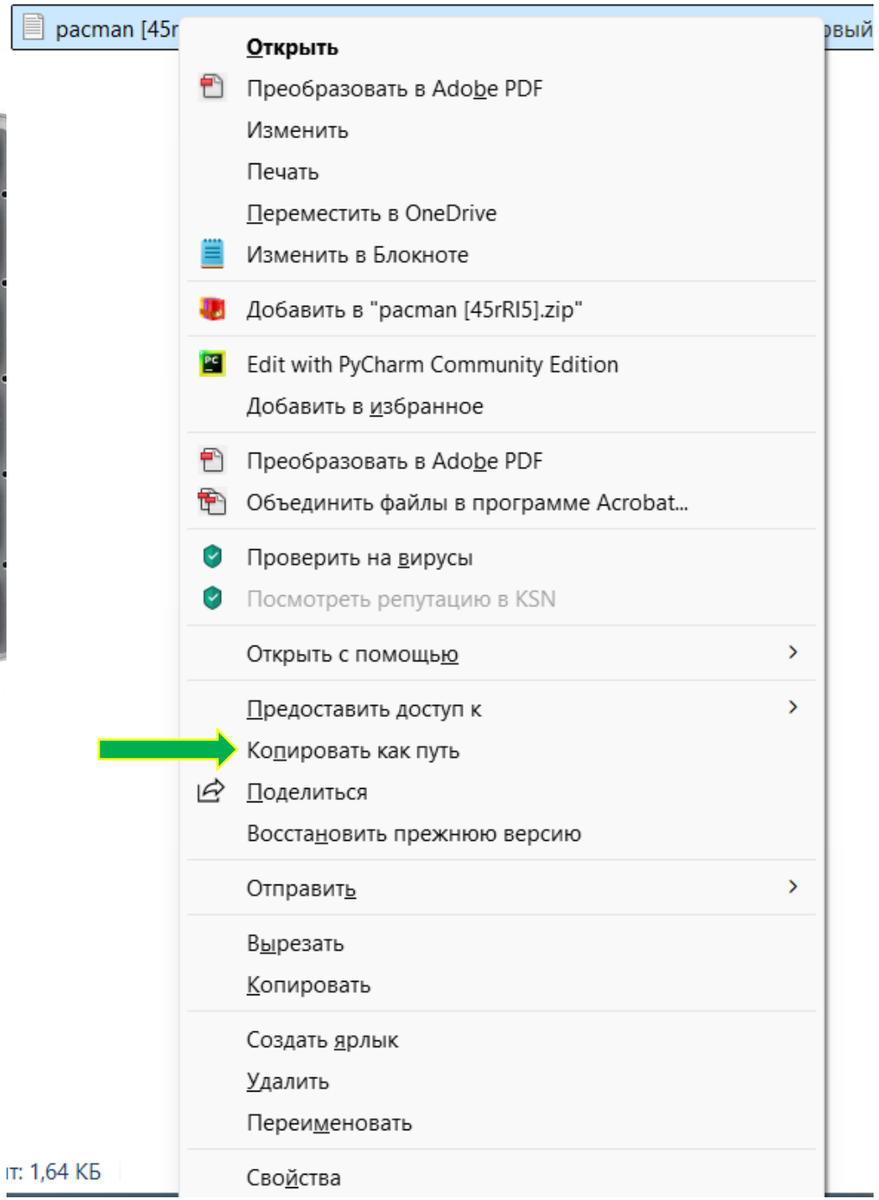
Абсолютный путь начинается с корневого каталога и указывает на точное местоположение файла, независимо от текущего каталога.

Пример: **"C:/Users/ИмяПользователя/Documents/файл.txt"**

2. Зажать **Shift** (не отпускать) и правой кнопкой мыши кликнуть по файлу, выбрать "**Копировать как путь**"



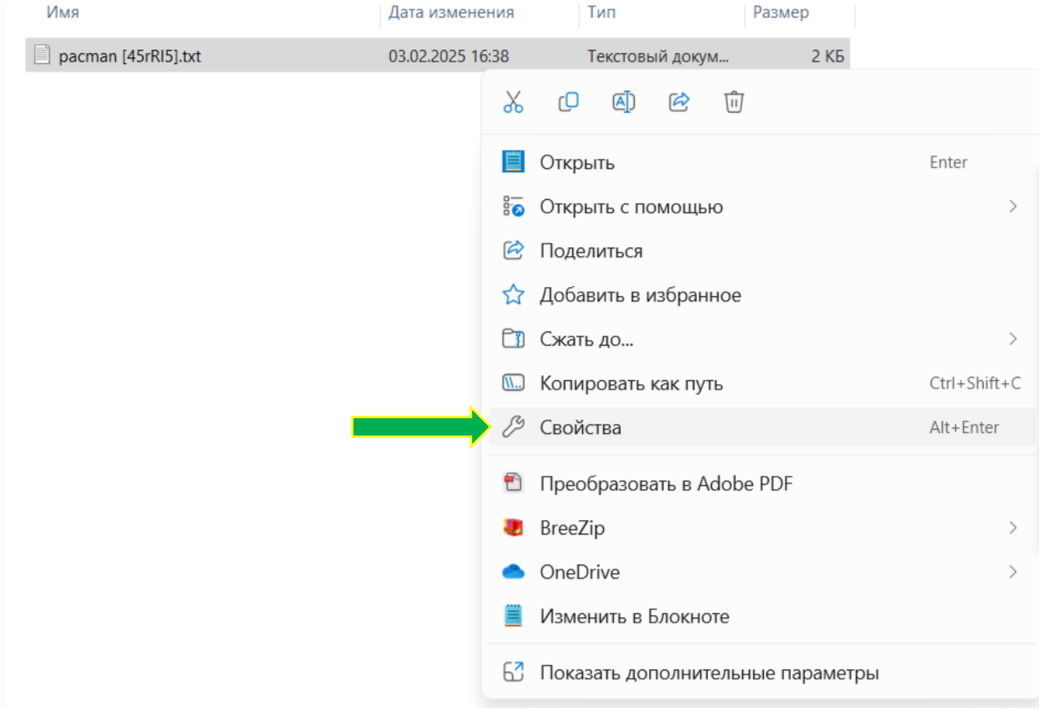


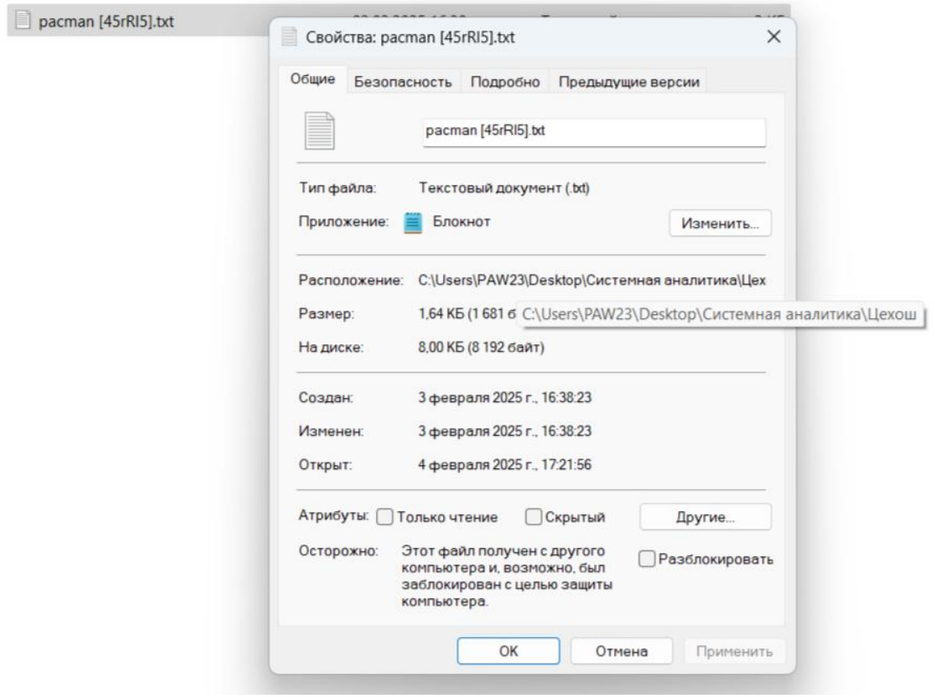


Когда вы пытаетесь скопировать полный путь к файлу через свойства файла в Windows, иногда название самого файла может не копироваться вместе с путем.

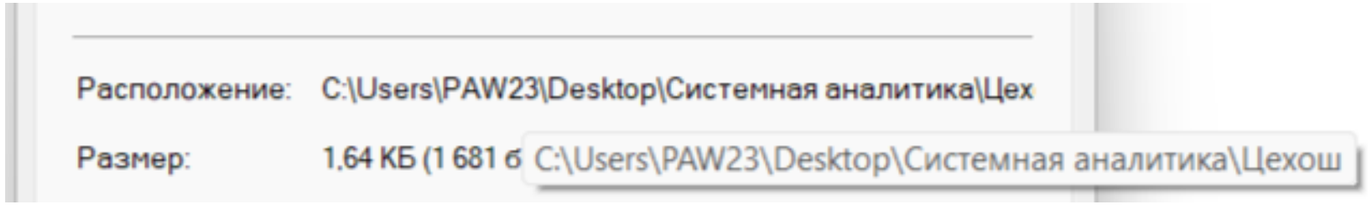
Вот несколько способов получить **полный путь с именем файла**:

1. Выбираем Свойства – расположение – копируем путь к папке, а затем **добавить имя файла вручную** (**если путь не полный**).





Эталон: "C:/Users/ИмяПользователя/Documents/**файл.txt.**



Абсолютный путь: 'C:\Users\PAW23\Desktop\ Системная аналитика\ Цехош**pacman [45rRI5].txt**'

Указывем абсолютный путь к файлу

file = open(**"C:\Users\SOFIA\Desktop\Системная аналитика\Цехош\pacman [45rRI5].txt"**, "r", encoding="utf-8")

Проблема :

В Python обратный слеш \ является специальным символом (используется для escape-последовательностей (\n, \t, \u и т.д.)).

Когда Python видит \U в пути, он пытается интерпретировать это как Unicode-символ.

Это вызывает конфликт при чтении путей и **создает 'unicodeescape' ошибку.**

Всегда используйте **прямые слеши** / в путях файлов.

Это работает и в Windows, и в Unix-системах.



file = open(**"C:/Users/PAW23/Desktop/Системная аналитика/Цехош/pacman [45rRI5].txt",** "r", encoding="utf-8")

Чтение содержимого файла

read() - читает весь файл как одну строку (считали все символы).

readlines() - читает файл и возвращает список строк (количество строк).

In [2]:
```
f = open('C:/Users/PAW23/Desktop/Системная аналитика/Цехош/pacman [45rRI5].txt', "r", encoding="utf-8")
text = f.read()
print(len(text))
```

```
1681
```

In [3]:
```
f = open('C:/Users/PAW23/Desktop/Системная аналитика/Цехош/pacman [45rRI5].txt', "r", encoding="utf-8")
text = f.readlines()
print(len(text))
```

```
41
```